Development Plan
TeLaC

Team 24, Maca Boca
Javayria Mudassar
Yilin Jiang
Nour Fawaz
Catherine Yang

Table 1: Revision History

Date	**Developer(s)**	**Change**
24-09-2026	All	Added first draft version
28-09-2026	All	Updated sections and reviewed document

The following document outlines the roadmap and development plan for improving and integrating a speech annotation tool into an existing TeleHealth platform. Included in the document are proprietary information (1-3), team logistics (4-6), project workflows and scheduling (7-9), and expected software tools and practices (10-12).

1 Confidential Information?

No confidential information is involved in this project. Any information provided from supervisors or stakeholders is trade-wide information and contains no trade-secrets or other confidential information requiring protection.

2 IP to Protect

There is currently no intellectual property requiring protection for this project, as the supervisor does not hold a patent related to the project.

3 Copyright License

For this project, the team will adopt the MIT copyright license. To summarize the license, any person may use, copy, modify, publish, distribute, sublicense or sell the software. However, the original copyright license must be included in any copies or significant utilizations of this software. Lastly, the project is provided in its final state, such that the team is not liable for damages or other claims resulting from its use. The full copyright license description can be found in the team repository under LICENSE.

4 Team Meeting Plan

Weekly meetings will be held between all members on Wednesday at 6:30PM, ideally in person; whether that's on campus or at a team member's home. This schedule is flexible to accomodate online meetings, if necessary. Members are expected to have read up on all relevant materials and completed any assigned action items from the previous meeting, to ensure that meets are productive and efficient. All members are encouraged to chair meetings, and meeting minutes will be taken by a designated team member, and shared with the team after each meeting.

Additionally, weekly meetings with the supervisor and other professors involved in the project will be held on Fridays, in order to stay on track with requirements and receive ongoing guidance throughout our deliverables. Each meeting will be recorded and meeting minutes will be taken as well with the same template as general meetings.

Meeting Minutes/Agenda Template

1. Date, Time, Location
2. Attendees
3. Goal of Meeting
4. Agenda items (Review of previous meeting minutes, discussion of any challenges the team has, questions/concerns, and required team decisions)
5. Action Items (Assign responsibilities to each member, note down deadlines and potential blockers between items)
6. **IF a supervisor meeting is upcoming**: note down all questions to be discussed with the supervisor, and any updates that we can provide them.
7. **IF a new deliverable was dropped**: review requirements as a team, brainstorm ideas/thoughts, and assign responsibilities to each member.
8. Notes (Summarize decisions, close meeting)

5 Team Communication Plan

Communication plays a vital role in ensuring all team members share a similar understanding of the project, each milestone is completed properly, and ideas, thoughts, and emergencies are conveyed/discussed. The following contains the different communication platforms that will be used, their purpose, and each team member's expectations regarding each form of communication.

5.1 Communication Tools

- **iMessage**: An iMessage group chat will be created to communicate with one another in a more convenient way. iMessage will strictly be used as a form of communication, mainly for casual conversation related to the project, discussing project details, and planning project meetings.
- **Google Drive**: A shared google drive folder will be created to house all documents relevant to the project. For example, brainstorming documentation and meeting minutes. It will be organized into folders each dedicated to a specific phase of time (in weeks) and project deliverable.
- **Teams**: A team group chat will be used for to conduct online meetings as well as share files not directly related to a specific project deliverable. For example, to share deliverable document examples and relevant course content pertaining to the project.
- **Email**: Email will be used to communicate with professors and supervisors who are onboard with the project.

- **GitHub**: A GitHub repository will be created to track project tasks using issues, which will be labeled depending on the task type. Additionally, a Kanban board will be created to track project task flow and completion by their assignees. The GitHub will also contain all the deliverable documents including the source code of the entire project.

6 Team Member Roles

Team member roles for this project will be divided into technical and administrative roles. From the initialization of the project, team members will be assigned all technical roles. This ensures that all members gain experience with development, testing, documentation, research, and review throughout the project. Technical responsibilities will be delegated according to individual tasks requirements, such that each team member may actively learn the skills needed for each technical role.
Next, each team member will initially be assigned one primary administrative role. These assignments are not fixed and may change throughout the project based on workload, individual expertise, and project needs. This structure allows the team to maintain clear ownership of administrative responsibilities while remaining flexible as the project evolves.

See Table 2 and Table 3 for each role, its responsibilities, and its corresponding assigned member(s).

Table 2: Technical Roles

Technical Role	**Responsibilities**	**Member(s)**
Developer	Implement end-to-end development of project features.	All
Documenter	Contribute to all project documentation. Document code in accordance with the project coding standard (See Section 12).	All
Tester&Reviewer	Ensure all features are tested and reviewed prior to implementation.	All
Researcher	Research domain knowledge and technical concepts required for project tasks. Apply research findings to software design and implementation.	All

Table 3: Administrative Roles

Administrative Role	Responsibilities	Member(s)
Liaison	Act as the primary communicator with professors, supervisors, and peer reviewers. Lead meetings with TAs and other external stakeholders when required.	Javayria M.
Administrator	Submits all team deliverables. Verify that required components are included in team documentation.	tbd
Coordinator	Take meeting notes and record feedback and action items. Ensures all members are assigned an equal task load.	tbd
Subject Matter Expert	Prepares all meeting agendas. Maintain project resources and bibliography. Primary researcher for required peer review knowledge.	tbd

7 Workflow Plan

Git Usage

The team will use Git and GitHub for version control and project management. After being assigned individual tasks, team members are required to create an issue, and to create and work on a new branch. All work will be completed on the branch and submitted for review through a pull request (PR). All members are encouraged to review any open pull request, but at least one team member must review and approve the changes before the user can merge their branch onto main.

Naming Conventions

The project will make use of following naming conventions:

Pull Requests/Commits: `<type>(<scope>): <short description>`. The supported types are:

- `feat`: Adding new functionality
- `fix`: Fixing a bug/error

- `docs`: Updating documentation
- `test`: Adding/updating tests
- `refactor`: Restructuring code, no external behavior change
- `chore`: Github maintenance, builds, dependencies

The scope identifies the primary component or document affected by the change, such as `ui`, `backend`, `data`, or the name of a specific document. The scope can be omitted if no appropriate scope applies.

For example:

```
feat(ui):  add activity selection screen
```

Branches: `<type>/<name>/<short-description>`, where `<type>` identifies the purpose of the branch, `<name>` identifies the contributor, and `<short-description>` briefly describes the work being completed. Descriptions should use lowercase words separated by hyphens.

For example:

```
feat/javayria/activity-selection-screen
```

Handling Pull Requests

When opening a pull request, the default template will be applied. It includes a description of the changes, added tests + test scenarios, linked issues, and a checklist. The checklist serves as a reminder for authors to review their work before submitting the pull request for review, but does not block it from being merged. When merging pull requests, the 'Squash and merge' option should be selected to help keep *main*'s history clean and readable.

Issue Management

All progress on the team will be tracked via Github Issues and a Kanban board. This includes every lecture and different type of meeting attended, as well as all development tasks and deliverables. Template issues have been created for each type of task/issue, and can be easily updated via configurable files in the repository; currently including sections for dates, descriptions, and relevant notes. Each issue also has a specific label to categorize it (e.g "Lecture" for Lecture Issues, and "Bug" all Development Bug Issues). Each member will be responsible for updating their own issues with relevant pull requests (must be closed by updating the 'Linked Issues' section of the PR description), and status changes as the task progresses. The coordinator will also regularly review the project board to ensure that all issues are on track.

Issue Templates:

1. Lecture Issue: To track any relevant notes detailed in lecture.

2. Supervisor Meeting Issue: To track all pre-meeting agenda items, questions, and notes with the supervisor and external teams.
3. Team Meeting Issue: To track all pre-meeting agenda items, questions, and notes for general meetings.
4. TA Meeting Issue: To track all pre-meeting agenda items, questions, and notes with the TA.
5. Deliverable Issue: To track requirements and progress on all deliverable items.
6. Blank Issue: To track any other important items.

CI/CD

To prevent any failures or issues from occurring, we'll be using Github Actions to implement our main CI/CD pipelines. These workflows will then be triggered on every pull request that opens or updates, and again on every merge to main. Each workflow will run the project's integration, contract or unit tests, and check for any formatting or linting errors before allowing the changes to be merged. If any of these builds or runs fail, the team member who opened the pull request must go back and fix the issue before merging. Another workflow will also be added to compile our LaTeX documentation whenever a `.tex` file is changed, ensuring that every document still generates a PDF without any errors. The compiled PDFs will also be uploaded as artifacts so whoever is reviewing the pull request can actually verify the final formatted output before approving. Once every check has passed, a final team member must stamp their approval in order to finally merge the changes into main.

Workflow Summary:

1. Brainstorm and discuss all tasks as a team.
2. Individual task is assigned to a team member, who creates an issue on the kanban board to track.
3. Team member creates a new branch from latest main, to work on their task. They must ensure that they're frequently pruning their branch to the latest main.
4. Any progress or significant changes should be updated on the kanban issue board.
5. Team member regularly commits their changes, with descriptive/clear commit messages that follow the outlined naming conventions.
6. Once the task is complete, the team member submits a pull request, and assigns a reviewer. The pull request description should include be complete, with all template fields filled out and checklists up to date.

7. The reviewer checks the PR, following any testing or tophatting instructions provided, and leaves comments if it needs any changes. If not, they approve the PR with a smiley face.
8. Once the PR is approved, the team member double checks their work and merges their branch into main.
9. Team member closes the issue on the kanban board if it was not done by an accompanying PR.

8 Project Decomposition and Scheduling

We will use GitHub Projects as the single source of truth for documentation and code (link to the GitHub project is HERE). The board (*Maca Kanba*) contains all project issues, with a Kanban view for day-to-day work and a table/roadmap view grouped by milestone. Every issue has at least one assigned owner to ensure it gets completed, though there may be multiple collaborators if required. Two issue types are used:

1. *Documentation issues*: one per deliverable section, using the documents.md template (Document Name / Description / Due Date / Reviewed by).
2. *Code issues*: one per feature/module/task, using a separate code template.

Milestones

Milestones are grouped by graded deliverable date to help keep the team on track for due dates. Peer-review milestones follow immediately after their corresponding deliverable. For example, if M2 is SRS + Hazard Analysis, then M2.1 is SRS Peer Review and all issues related to addressing feedback are tracked there. Code-heavy sections (POC, Rev 0 module implementation, Rev 1) may get their own milestones separate from any documentation due the same week, since code issues require a separate lifecycle that relies more on testing and thorough reviews.

Project Schedule

- **September 28 – November 19: Requirements**
  - Problem Statement
  - Proof of Concept Plan
  - Development Plan & Team Charter
  - Requirements Document (SRS)
  - Hazard Analysis, V&V Plan Revision 0
  - Design Document Revision -1 with CD-Fall

- **January 18 – January 29: Proof of Concept & Module Implementation**
  - Design Document Revision 0
  - Scheduling constraint requests
  - Core module implementation
  - POC Demonstration
  - Team Contribution Survey for Revision 0
  - Team Productivity Report for Revision 0
- **(POC/Rev 0 Demo dates) – March 8: Integration & Verification**
  - Integration of completed modules
  - V&V Report
  - CD-Winter
- **March 8 – April 6: Finalization & Expo**
  - Team Contribution Survey for Revision 1
  - Team Productivity Report for Revision 1
  - Source code and Final Documentation for Revision 1
  - Final Demonstration
  - Capstone EXPO

The project can thus be broken down into the following milestones:

Table 4: Project Milestones

Milestone	Deliverable(s)	Due Date
M1	Problem Statement, POC Plan, Development Plan, Team Charter	Sep 28
M1.1	Peer Review: above	Oct 1
M2	SRS + Hazard Analysis	Oct 19
M2.1	Peer Review: SRS + Hazard Analysis	Oct 22
M3	V&V Plan Rev 0	Nov 2
M3.1	Peer Review: V&V Plan	Nov 5
M4	Design Doc Rev -1, CD-Fall	Nov 16
M4.1	Peer Review: Design Doc Rev -1	Nov 19
M5	Scheduling constraint requests	Jan 18
M6	Module Implementation	Jan 21
M6.1	Design Doc peer module reflection	Jan 25
M7	Team Contribution Survey + Productivity Report Rev 0	Jan 29
M8	V&V Report + CD-Winter	Mar 8
M9	Team Contribution Survey + Productivity Report Rev 1	Mar 18
M10	Final Documentation (Rev 1) + Source Code	Mar 31
M11	EXPO Demonstration	Apr 6

9 Proof of Concept Demonstration Plan

A high-level overview of our proof of concept demonstration:

1. A patient will have previously recorded a bilingual assessment session on the TeleHealth platform.
2. The clinician will log in and select the same patient profile and language assessment session from the patient's history.
3. The SAFA interface retrieves a transcript of the recording, divided into time-coded chunks with clinically meaningful markers (e.g., language switches, utterances, and assessment tasks).
4. The clinician reviews and annotates the transcript, verifies the markers, and labels key details about the speaker and their performance to support their assessment of the patient's language abilities.

The high-level plan above does not fully address the major risks that could arise during the project, but we have identified ways to address these issues.

- By failing to properly **integrate SAFA into the TeleHealth platform**, clinicians will have to leave the web platform and transfer files manually

to SAFA, defeating one of the project's main purposes. The main concern is that SAFA was built as a local, standalone tool, whereas TeleHealth exists as a prototype web-based platform with its own architecture and design.

  - To address this, the POC will demonstrate an end-to-end workflow in which clinicians access a patient's recorded assessment session within the TeleHealth platform and use SAFA's core modules. The usability and performance of each module, including chunk editing, video playback, and annotation markers, will be compared with its performance in the standalone SAFA tool to verify successful integration.

- Another large risk is failing to **keep the video and transcript synchronized in the browser**. Clinicians might make edits to the wrong sections of the transcript and become confused by the timing. SAFA depends on the video and transcript staying aligned so clinicians can make precise edits. Streaming video over the web can also add delay and buffering that may not occur when running SAFA locally, which could contribute to synchronization issues. This will require extending TeleHealth's Media Processing module to stream video and seek to specific timestamps, rather than loading the full file at once.

  - To address this, the POC will play a full-length recorded session (at least 10 minutes) and repeatedly jump between sections from the start, middle, and end of the video. We will check that the highlighted chunk matches what is being said and measure the gap between the video's timestamp and the transcript's timestamp. As a tentative target, we can aim to keep this gap under half a second with no noticeable drift throughout the demonstration.

- The system may also struggle to properly **categorize and pre-annotate important parts of the transcript**. If this happens, clinicians may still need to manually search through entire recordings for key moments and markers, reducing the time savings that the transcription and annotation modules are intended to provide.

  - To address this, the POC will compare the system's automatically generated markers and tags against a manually reviewed sample of the recording. We will measure how accurately the system identifies speaker turns, language changes, assessment tasks, and significant moments, as well as how much time these markers save during clinician annotation.

- Finally, one of the largest risks is failing to **protect patient data**. Unauthorized users may gain access to assessment videos, transcripts, and annotations. Since these files will live in the clinician's TeleHealth account, unauthorized access could result in a serious security failure and expose sensitive information about children.

– To address this, the POC will use TeleHealth's existing authentication to show that a clinician can only access recordings and annotations for their own patients. We will also verify that patient data is encrypted during transmission and stored securely. This will be demonstrated by logging in as a second clinician account and confirming that it cannot view or edit the first clinician's data, including through direct links or API requests.

If the team is unable to make progress on the risks above, it could lead to minor or major consequences for the project. Addressing these risks early will help the team confirm that the project is feasible and define clear metrics for success.

10 Expected Technology

The technologies listed in this section are expected to be used within this project, but are not firm requirements or bounding limitations on technology that can be used. The project integrates SAFA, an existing Python-based annotation tool, into TeLaC, an existing telehealth platform. The main technology constraints are therefore set by these two systems, as shown in Table 5.

Table 5: Expected Technologies

Category	**Expected Technology**
Programming languages	Python (SAFA modules and integration), TypeScript and JavaScript (TeLaC and the SAFA user interface)
Frameworks and libraries	Node.js and React.js (existing TeLaC stack); React for the SAFA user interface; existing TeLaC services (Amazon S3 for video storage, Google MediaPipe, Deepgram)
Video relay service	An event-driven AWS service (Lambda, DynamoDB, and SQS). We expect to implement this ourselves
Pre-trained models	No models will be trained by us. SAFA uses OpenAI's Whisper for draft transcripts
Linters	Ruff (Python), ESLint (TypeScript/JavaScript)
Unit testing	pytest (Python), Jest (TypeScript/JavaScript)
Continuous Integration	GitHub Actions
Development tools	Visual Studio Code, Git, GitHub, and GitHub Projects

11 Use of AI and LLMs

11.1 How AI and LLMs Will be Used

Large language models will be used as a tool to check and generate technical writing and code.

11.1.1 Generating Writing and Code

Team members are permitted to use any of the AI tools listed in the Tools subsection to generate technical writing and/or code to complete project related tasks. However, any output used must be strictly verified as outlined in the Verification of LLM Output section.

11.1.2 Checking Writing and Code

Before a team member creates a pull request to merge their work with the main branch, it is highly recommended and enforced where the writer deems it necessary to check their work against an LLM.

For written deliverables, the team member may ask the LLM whether their writing covers the "Exceeds" criteria in the corresponding rubric posted on avenue. This includes asking for idea generation, fixing grammatical mistakes, and enhancing the writing structure and tone.

As for code, the member may ask the LLM whether their code contains any errors and follows the code standard outlined in Section 12.

11.2 Verification of LLM Output

In order to verify the output of the LLM, every line of code and writing must first be reviewed and completely understood by the individual generating the output of the LLM. For example, if the team member uses an LLM to generate code they must be able to explain to another member what the code is doing. Additionally, if they generate writing, they must understand what the writing means. If the team member is unable to do so, they must revise their section such that they are confident in their work and can clarify what is being done. This goes in hand with using an LLM as a checking tool.

Next, each team member's section pertaining to all project tasks, including those generated or checked with an LLM, will be reviewed by another group member. For written sections, the reviewing member will check whether the writing is clear, concise and understandable. As for code, the member will determine if it is clean, readable, maintainable, and if it follows the code standard outlined in Section 12.

11.3 Tools

All team members are permitted to use the following AI tools to check and generate writing and code as they see fit:

- GitHub Copilot
- Claude
- ChatGPT

12 Coding Standard

The coding standards used by this project will be aligned with the existing standards of the system to maintain consistency with previously developed components. For example, an existing module, TeleHealth, followed the ReactJS guidelines provided by Pillar Studio for React development and PEP 8 for Python development. Where applicable, this project will follow the same established standards. Any additional programming languages used during development will follow the established coding conventions and style guidelines recommended by the language creators or maintainers.
The codebase will also follow a consistent structure and design pattern throughout the repository. For example, if an atomic folder structure is established, all team members will follow the same structure when adding or modifying files.

13 Appendix — Generative AI Use Disclosure

13.1 AI Tools Used

- Claude Sonnet 5 (Medium)

13.2 How and Where Used

- Claude was used to review sections of the document against the rubric requirements and the specific questions written in each section's prompt. This was applied in both the Development Plan and Problem Statement documents.
- Claude was used to improve the wording, clarity, and flow of written sections, including reducing repetitive phrasing and correcting grammar. This was applied in both the Development Plan and Problem Statement documents.
- Claude was used to help identify and fix LaTeX compilation errors. This applies to the following sections of the Development Plan: 6, 8, and 11.

13.3 Verification of AI-Generated Content

To verify the output of the LLM, each team member went over their section and presented their work to the group, explaining what they had completed since the previous meeting. Each member was able to successfully explain their sections. Additionally, before any pull request was approved, a team member who had not written the section reviewed the work to ensure it was cohesive, accurate, and consistent with the rest of the document. Considering this document contains no factual claims, the verification of the output was mainly assessing whether it was consistent with the rest of the document, rather than fact checking.

Appendix — Reflection

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. Why is it important to create a development plan prior to starting the project?
2. In your opinion, what are the advantages and disadvantages of using CI/CD?
3. What disagreements did your group have in this deliverable, if any, and how did you resolve them?

Javayria

If there's one thing I took away from my co-op, it's that structure and planning before starting a project is extremely necessary. We are all software engineers with our own ways of thinking and working, and while individuality is great, there needs to be some pre-agreed upon, formalized set of rules and methodologies for the team to succeed. Having a plan allows the team to focus on what's important rather than getting lost in fine-grained details. It also sets a standard, where everyone is aware of the level of work quality expected of them beforehand, so they can keep that in mind while working. Is it difficult to confront a team member who isn't contributing to the level expected of them? Yes, it is. However, that's much less likely to happen if the conversation happens at the very beginning, since it helps the team find common ground and ensures everyone goes into the project with the same mindset. I feel confident that my team members and I will benefit from having this plan in place as the project grows more and more complex.

One of the biggest advantages of CI/CD is that it helps catch mistakes and issues with the code before it goes to production (or in this case, the main branch). Running and troubleshooting builds manually is an extremely frustrating process (as any software engineer can tell you!), so having a workflow run a linter or unit tests against new changes before they're ready to be merged is a great first step for verification. One of the major drawbacks, though, is the

amount of time it takes to both run and troubleshoot CI/CD pipelines (here, we mean in the context of GitHub). It's very easy to find yourself waiting on a 30-minute build pipeline, only for it to fail midway and need to be run again. This can slow down the coding process and cause PRs to take multiple days to be reviewed and merged. I expect that we will need to tweak/update the workflows throughout the term as we notice certain parts taking too long or repeatedly failing.

Our team did not have any major disagreements when working on this deliverable. I found that everyone was respectful and open to hearing each other's ideas, which is a good sign for the project's future. As we start reviewing each other's work moving forward, we will need to continue being respectful and kind in our communication. If there happen to be any disagreements within the group, then both parties are expected to handle them immediately in an open, respectful manner.

Catherine

Without a development plan to layout the foundation for how our project is structured and run, I think we'd immediately lose our heads when it comes to things like formatting, timelines, and most *importantly* motivation. When things get messy and convoluted, I personally dread tackling it and putting off work as important as our capstone project is the last thing I want to do. The plan is there to make things easier for ourselves as members, as it's just a structured guideline that we can continuously refer to throughout the project. We don't have to continuously reinvent new rules or reconvene to discuss things that we've already done and gone over. As the project progresses, I'm 100% sure things will only become more complex and messy, so having a plan in place from the start is even more important to ensure that we're moving as efficiently and as organized as possible.

I believe that there's a lot of pros to having CI/CD integrated in your project. If there's any test-breaking code, linting errors that create rendering or deployment errors, or stale code from bad merges/previous commits; having a pipeline to catch everything *before* they go into main can ultimately save a lot of headache everybody involved. As now maintenance can happen while the new changes are contained in a pull request. On the downside though, they can take a while to setup. I expect that we'll have to set aside additional time for the testing process of any additional pipelines we want to create.

Finally, I don't believe we had any team disagreements for this deliverable. We had a pretty straightforward and productive meeting on our next steps, brainstorming collectively for each section item, before we individually assigned tasks and broke off to work on them separately. We agreed though that if we have any future disagreements, the members involved would communicate respectfully and sort it out together before moving forward with any work.

Yilin

For myself, I think of the development plan as the underpainting of any project. Starting a painting without one is always more intimidating, unstructured, and difficult to truly begin making progress. In the same way, a development plan allows both myself and my team members to sit down and discuss our understanding of the project and our expectations for both the journey and the outcome. It is allocated time to build team communication and understand both the problems we will be facing and what we hope to solve.

I believe one of the advantages of CI/CD is the significant increase in consistency and discipline it can provide when developing software. Automated testing, builds, and other checks can help reduce human error by acting as a safety net and providing immediate feedback. On the other hand, one of the downfalls of CI/CD is that it can give you a false sense of security. For example, under a time crunch, the process of thoroughly going through a merge request might be rushed, or developers may rely too heavily on automated tests that do not catch issues a human may catch, leading to serious consequences.

Lastly, the group was very determined and organized for this deliverable and had no disagreements. As we were all excited about finally nailing down a project, our subsequent meetings led us to divide up tasks to meet this deliverable. I think the open communication and structured planning that came from our struggles getting a project actually helped us stay conflict-free during this deliverable.

Nour

I believe having a development plan really helps build a foundation for the entire project. It allows all the team members to develop the same general understanding of what the project entails and what is expected of us. I believe without the plan, we would be confused about where to start, what resources we'd need, and what steps to take to make our project smoother and more manageable. After creating the development plan, I have a much better understanding of the tasks we'll likely be doing and the tools we'll need.

Some advantages of implementing a CI/CD pipeline are, first, in the long run, it's faster and helps automate a lot of processes you no longer have to worry about manually. Second, it helps catch bugs early, which is much more preferable to resolving them later. Third it helps maintain consistent quality across all builds throughout the project. For example, a defined set of tests will run automatically every time a build enters the pipeline. There are some disadvantages, however, and the one I find most concerning is the initial learning curve. To implement an efficient and truly beneficial CI/CD pipeline requires an understanding of how to construct one, and since I haven't built one before, this will take time to learn. In addition to this, the pipeline needs to be maintained as the project progresses, which also requires dedicated effort. This might not

sound like much, but when you're focused on actually completing the project, it's easy to overlook things like this.

Fortunately, our group did not experience any disagreements when completing this deliverable. Prior to starting, we had a meeting to discuss task delegation and task deadlines. Additionally, we set up the entire project environment to prevent any member from running into issues later. Since expectations and responsibilities were made clear from the start, there were no conflicts regarding next steps or task completion timelines.

Appendix — Team Charter

External Goals

For this capstone project, our team's external goals are to achieve an A+ first and foremost, but to also have a finished product that actually demonstrates what we've learned about healthcare in a speech-language pathology setting, and also how technology can actually reduce barriers that many in the field face, from both clinicians and patients. We hope to have a product that future employers can utilize, and also showcase on our resumes to show off our skillset as engineers.

Attendance

Expectations

All team members are expected to arrive on time to pre-agreed meetings and remain for the entirety of the scheduled time slot. Team members are expected to attend all meetings unless they notify the group in advance that they are unable to attend. Notice should be provided at least one day before the meeting whenever possible. If a team member is unable to attend a meeting, they are responsible for reviewing the meeting notes, communicating with the team as necessary, and completing their assigned work before the next meeting. Missing more than three meetings will result in a meeting with the rest of the team to discuss next steps. Lastly, team members are expected to arrive prepared, with any technical or administrative tasks assigned prior to the meeting completed.

Acceptable Excuse

Acceptable reasons for missing a meeting include unexpected emergencies, health-related circumstances, and anticipated absences. Emergency circumstances will be addressed with consideration for the situation. Team members should communicate any health-related absence or anticipated impact on their availability, such that the team can adjust the workload. Anticipated absences should be communicated before the meeting is scheduled, or as soon as the member becomes aware of the conflict.

In Case of Emergency

If an emergency does occur, the member does not need to alert the rest of the team. They can do so afterwards when reasonably able. The team will bring flowers and do their best to console the team members. In the case that the emergency coincided with a deadline, the affected team member may make up their contribution through future tasks.

Accountability and Teamwork

Quality

Team members are expected to spend 20-30 minutes reviewing relevant documents before each meeting, or enough to contribute meaningfully and raise questions or notes as needed. Before meetings with the supervisor, all members must be fully aware of the project's status and prepared to both ask and answer questions.

For deliverables, each member is responsible for completing their assigned section to the best of their ability: original content, free of spelling and grammar errors, and going beyond the bare minimum. If a section is entirely AI-generated, the author must be able to explain the writing/code and prove to have a deep understanding of the work they put out. All team members are responsible for properly testing their code with the appropriate endpoints to ensure any existing functionality is not broken. Each section should be checked against the Avenue rubric to confirm all requirements are met, and reviewed by a designated team member before submission.

All deliverables must be finalized two days before the official deadline to allow time for review. Any issues found afterward are to be fixed by the original author. Final submission occurs by 7:00 pm on the due date to allow time for technical issues.

Attitude

Code of Conduct
The following code of conduct was developed by the team and outlines what is expected in terms of attitude, behaviour, cooperation, and contribution from each team member.

- **Communication**: All team members are expected to communicate with one another in a kind and respectful manner. They should approach all conversations with an open-mind and be willing to take constructive feedback when needed. Additionally, all team members are expected to respond to group messages within approximately 1-2 hours of being directed at them.
- **Positive Attitude**: All team members are expected to approach tasks with a positive attitude and outlook to maintain team morale and encourage one another.
- **Collaboration**: All team members are expected to communicate their thoughts and ideas, participate thoroughly in group meetings by providing feedback and constructive criticism, and cooperate efficiently and effectively with other members.
- **Task Completion**: All team members are expected to have their tasks completed by the agreed upon deadlines. Additionally, group members

must remain proactive and help other group members who are struggling with their tasks.

- **Dedication to Quality**: All group members are expected to complete their tasks with the utmost quality and to the best of their abilities.

Conflict Resolution Plan
When experiencing conflict, the following guide will be used:

1. The team will hold a group discussion allowing each team member to share their perspective in order to pinpoint the root of the problem
2. The team will brainstorm potential solutions to the problem by finding common ground through shared goals and interests.
3. The team will develop a solution plan that all opposing parties are satisfied with.
4. The team will begin implementing the plan and monitoring its progress. If the solution yields no results, the team may repeat the process until a proper solution is reached.

Stay on Track

Meetings
Mandatory in person meetings will be held approximately 1-2 times per week, depending on the current project milestone, to:

- Discuss overall current project progress
- Give each group member the opportunity to provide an update on the tasks they have been completing since the last group meeting
- Delegate tasks regarding the current project milestone
- Discuss upcoming TA check-ins to determine what topics should be brought up and what questions to ask

GitHub
Github issues will be used to break down project millstones into tasks that can be delegated to group members. Each member will be assigned a task, which they are expected to update on the project Kanban board to communicate its progression to the rest of the group.

Course Google Calendar
Each member will use the course Google Calendar to stay aware of upcoming lectures, deadlines, and TA meetings. This will allow the team to prepare any questions and concerns regarding the project in advance.

Deadlines
All group members are expected to have their parts of the project deliverable completed and on their GitHub branch at least one day prior to the deliverable's deadline. The team will then meet the following day to assemble the document, ensuring it is ready for submission at least 6 hours before the deadline.

Rewarding Team Members
Members with exceptional performance will be rewarded by having their hard work and dedication acknowledged by the team. This may also be rewarded by taking on more leadership roles, provided they are open to doing so. Additionally, other members are free to reward them with snacks.

Target Metrics and Consequences
Members who are performing below expectations will be confronted by the team in a kind and supportive manner. They will be asked about the cause of their reduced performance, and the team will work with them to address the issue constructively. However, if the member remains unresponsive the team will follow the conflict resolution plan outlined in the Attitude section. If all else fails, the issue will be raised with the TA and, if necessary, the course instructor.

- **Attendance**: Team members are expected to attend all in-person group meetings. If they are unable to attend in person, they must join online unless they have an emergency. If a team member misses 3 group meetings, they must purchase a tube of milk chocolate M&M's minis for each team member.
- **Commits**: The number of commits expected from each team member should reflect the amount of work they were assigned. However, each member is expected to make at least 5 commits per project deliverable, with every commit contributing meaningful value to the project. Failing to meet the minimum requirement will result in the member refilling the rest of the group's water bottles for an entire day when asked to do so.
- **GitHub Issues**: Team members are expected to have all their issues 100% completed (moved to the "Done" section of the Kanban board) one day prior to a project deadline. Failing to do so will result in the member purchasing a bag of milk chocolate M&M's for each team member.

If a group member misses the target metrics outlined above more than twice, the team will first meet with them to discuss the behaviour. If the member's behaviour persists and deadlines continue to be missed, the issue will be raised with the TA and, possibly, the course instructor. This also applies if a group member consistently fails to contribute fairly to the project, which can be analyzed via their completed issues and number of commits.

If a group member consistently reaches the target metrics early, they will be given the choice of the next meeting's location, or the ability to set restrictions

on where it will be held.

Team Building

For group meetings, first they will be held at certain locations favoured by different team members, such as Coco's for boba tea, so that our time together feels more like a hangout rather than simply meeting to do project work. Next, members will bring snacks to stressful meetings to lighten the mood and spread joy. Lastly, the team has decided to hold all team meetings in person unless a team member is truly unable to attend.

Decision Making

All group decisions will be made by reaching a consensus, rather than a vote. This ensures that all group members are satisfied with the outcome of the decision.

The team will follow the process below to handle disagreements:

1. Each member will be given a turn to share their opinion and thoughts on the matter, while maintaining a positive attitude as outlined in the Attitude section.
2. After hearing everyone's perspective, the team will hold a short, project-focused discussion to work toward an agreed-upon consensus. Relevant technical documentation may be referenced during this discussion to ensure that project goals continue to be met.
3. If aconsensus is still not reached, a neutral third party, which can include an uninvolved team member or an individual onboard with the project, may facilitate the discussion or offer their perspective to help the group reach an agreement.